



UNIVERSITY OF
CAMBRIDGE



Institute of
Computing for
Climate Science

University of
Kent



Programming
Languages and Systems
for Science laboratory

Modelling and Verification of Climate Models

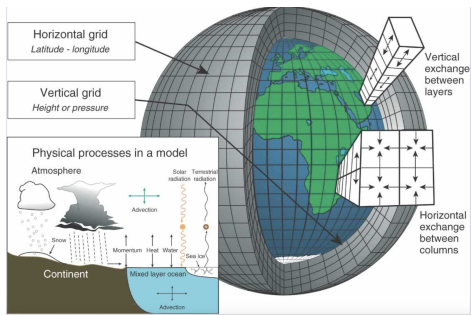
Chinmayi Baramashetru¹ **Dominic Orchard¹ & ²**

¹University of Kent, UK

²University of Cambridge, UK

November 6, 2025

Global Circulation Models (GCM)



Source: Edwards (2011)

- GCMs are large-scale models that simulate Earth's climate over time.
- They use physical laws, numerical methods, and observational data.
- Implemented as large Fortran codes running on supercomputers.
- Composed of interacting **submodels**
 - Each submodel runs on different grids, resolutions, and time steps.

Software Architecture of Climate Models

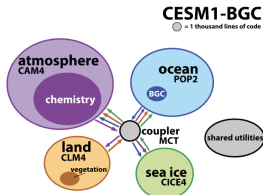


Figure 1. Architecture diagram for CESM1-BGC.

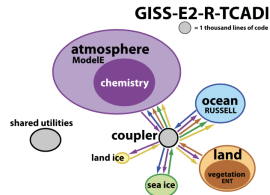


Figure 3. Architecture diagram for GISS-E2-R-TCAD1.

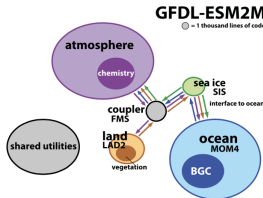


Figure 2. Architecture diagram for GFDL-ESM2M.

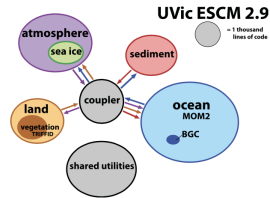


Figure 4. Architecture diagram for UVic ESCM 2.9.

Source: *The software architecture of climate models: a graphical comparison of CMIP5 and EMICAR5 configurations* – Alexander & Easterbrook (2015)

Software Architecture of Climate Models

Model interaction is coordinated by a software called **Coupler**

Role of Coupler

- Mediates data exchange between submodels.
- Performs **field exchange**, **interpolation**, **conversion between grid formats**, and **time synchronization**.
- Critical for reproducibility, scalability, and correctness in multi-component simulations.
- Minor errors in couplers can propagate and distort results.

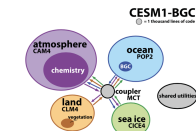


Figure 1. Architecture diagram for CESM1-BGC.

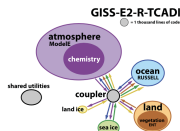


Figure 3. Architecture diagram for GISS-E2-R-TCAD1.

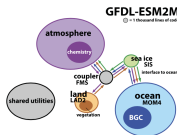


Figure 2. Architecture diagram for GFDL-ESM2M.

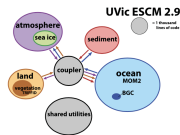


Figure 4. Architecture diagram for UVic ESCM 2.9.

Source: Alexander & Easterbrook (2015)

Coupler Correctness and Verification

- Verification efforts in climate models face a key challenge:
 - Verifying full models is extremely complex,
 - Couplers represent a **tractable verification target** — critical and shared.
- Couplers are **complex and loosely specified** software orchestration layers.
- Hard to ensure correctness, reproducibility, and maintenance.
- Widely reused across models (e.g., over 130 in major CMIP¹ projects).
- **High-impact target**: verification benefits many models at once.
- Similar to software components - contracts, modular reasoning, etc.

¹Coupled Model Intercomparison Project, a standardised framework for evaluating climate models.

State-of-the-art Formal Verification Techniques

- **Testing:** Used for regressions and quick feedback; no completeness guarantees.
- **Static Analysis:** Type systems, dataflow analysis, abstract interpretation etc.
- **Deductive Verification:** Formal conformance of source code to specification using expressive logic.
- **Proof Assistants:** Produces machine-checked proofs.
 - **Workflow:** Specs $\rightarrow$ formal model/code $\rightarrow$ tactics/automation $\rightarrow$ checked proof
- **Model Checking:** Explicit, symbolic, bounded; exhaustive state exploration against specifications.

State-of-the-art Formal Verification Techniques

- **Testing:** Used for regressions and quick feedback; no completeness guarantees.
- **Static Analysis:** Type systems, dataflow analysis, abstract interpretation etc.
- **Deductive Verification:** Formal conformance of source code to specification using expressive logic.
- **Proof Assistants:** Produces machine-checked proofs.
 - **Workflow:** Specs $\rightarrow$ formal model/code $\rightarrow$ tactics/automation $\rightarrow$ checked proof
- **Model Checking:** Explicit, symbolic, bounded; exhaustive state exploration against specifications.

State-of-the-art Formal Verification Techniques

- **Testing:** Used for regressions and quick feedback; no completeness guarantees.
- **Static Analysis:** Type systems, dataflow analysis, abstract interpretation etc.
- **Deductive Verification:** Formal conformance of source code to specification using expressive logic.
- **Proof Assistants:** Produces machine-checked proofs.
Workflow: Specs $\rightarrow$ formal model/code $\rightarrow$ tactics/automation $\rightarrow$ checked proof
- **Model Checking:** Explicit, symbolic, bounded; exhaustive state exploration against specifications.

State-of-the-art Formal Verification Techniques

- **Testing:** Used for regressions and quick feedback; no completeness guarantees.
- **Static Analysis:** Type systems, dataflow analysis, abstract interpretation etc.
- **Deductive Verification:** Formal conformance of source code to specification using expressive logic.
- **Proof Assistants:** Produces machine-checked proofs.
 - **Workflow:** Specs $\rightarrow$ formal model/code $\rightarrow$ tactics/automation $\rightarrow$ checked proof
- **Model Checking:** Explicit, symbolic, bounded; exhaustive state exploration against specifications.

State-of-the-art Formal Verification Techniques

- **Testing:** Used for regressions and quick feedback; no completeness guarantees.
- **Static Analysis:** Type systems, dataflow analysis, abstract interpretation etc.
- **Deductive Verification:** Formal conformance of source code to specification using expressive logic.
- **Proof Assistants:** Produces machine-checked proofs.
 - **Workflow:** Specs $\rightarrow$ formal model/code $\rightarrow$ tactics/automation $\rightarrow$ checked proof
- **Model Checking:** Explicit, symbolic, bounded; exhaustive state exploration against specifications.

Verification Techniques Used in Climate Models

- Testing is widely used.
- Other verification efforts (e.g., Interactive theorem provers)

A Coq Formalization of Taylor Models and Power Series for Solving Ordinary Differential Equations

Sewon Park 

Graduate School of Informatics, Kyoto University, Japan

Holger Thies 

Graduate School of Human and Environmental Studies, Kyoto University, Japan

A Verified ODE Solver and the Lorenz Attractor

Fabian Immler¹ 

Wave Equation Numerical Resolution: A Comprehensive Mechanized Proof of a C Program

Sylvie Boldo · François Clément ·
Jean-Christophe Filliâtre · Micaela Mayero ·
Guillaume Melquiond · Pierre Weis

Received: 19 April 2017 / Accepted: 27 December 2017 / Published online: 22 January 2018
© The Author(s) 2018. This article is an open access publication

Received: 12 December 2011 / Accepted: 23 June 2012 / Published online: 14 August 2012
© Springer Science+Business Media B.V. 2012

Abstract A rigorous numerical algorithm, formally verified with Isabelle, certifies the computations that Tucker used to prove chaos for the Lorenz attractor. The formalization is based on a formalization of a diverse variety of mathematics. Formalized mathematics include ordinary differential equations and Poincaré maps. The algorithm includes low level approximation schemes based on Runge–Kutta methods and interval arithmetic. On a high level, reachability analysis is guided by static hybrid

Fortran Dominance in Climate Modeling!

Aspect			
Current Fortran Usage in Weather/Climat			
WMO Compliance Requirements			
Performance Comparison (Fortran vs Pyt			
Migration Complexity	High - Years of effort, specialized expertise required	Very High	Very High
Skills Availability	Critical shortage - 1,050 Fortran jobs vs 83,591 Python jobs	Very High	High
Training Requirements	Months to years for scientists to learn new languages	High	Medium
Legacy Code Maintenance	Expensive - Millions of lines of tested, validated code	Very High	Very High
Interoperability Solutions	Fortran-Python interfaces available but add overhead	Medium	Medium

- Full migration to Python costs \$15 million
- Complete rewrites or migrations are risky: bit-for-bit fidelity, validation, and legacy code complexity.

Source: Uy, J. (2024)



Fortran Dominance in Climate Modeling!

Aspect			
Current Fortran Usage in Weather/Climat			
WMO Compliance Requirements			
Performance Comparison (Fortran vs Pyt			
Migration Complexity	High - Years of effort, specialized expertise required	Very High	Very High
Skills Availability	Critical shortage - 1,050 Fortran jobs vs 83,591 Python jobs	Very High	High
Training Requirements	Months to years for scientists to learn new languages	High	Medium
Legacy Code Maintenance	Expensive - Millions of lines of tested, validated code	Very High	Very High
Interoperability Solutions	Fortran-Python interfaces available but add overhead	Medium	Medium

- Full migration to Python costs \$15 million
- Complete rewrites or migrations are risky: bit-for-bit fidelity, validation, and legacy code complexity.

Source: Uy, J. (2024)



- Fortran lacks modern **formal verification tools** (e.g., Frama-C, KeY) available for C/Java.
- Tooling gap limits deductive reasoning and automated correctness proofs.

Fortran Dominance in Climate Modeling!

Aspect			
Current Fortran Usage in Weather/Climat			
WMO Compliance Requirements			
Performance Comparison (Fortran vs Pyt			
Migration Complexity	High - Years of effort, specialized expertise required	Very High	Very High
Skills Availability	Critical shortage - 1,050 Fortran jobs vs 83,591 Python jobs	Very High	High
Training Requirements	Months to years for scientists to learn new languages	High	Medium
Legacy Code Maintenance	Expensive - Millions of lines of tested, validated code	Very High	Very High
Interoperability Solutions	Fortran-Python interfaces available but add overhead	Medium	Medium

- Full migration to Python costs \$15 million
- Complete rewrites or migrations are risky: bit-for-bit fidelity, validation, and legacy code complexity.

Source: Uy, J. (2024)



- Fortran lacks modern **formal verification tools** (e.g., Frama-C, KeY) available for C/Java.
- Tooling gap limits deductive reasoning and automated correctness proofs.

CamFort

Our Approach

Major Coupler Frameworks



CPL6 and CPL7 couplers



Earth System Modelling Framework



Oasis Couplers



Modelling Coupling Toolkit

Core Functions of Couplers (Our Analysis)

- Couplers vary in design:
 - Standalone vs. embedded libraries
 - Different data exchange, timing, and interpolation
- Evolved over decades with limited specification.
- Based on a detailed review of major couplers, these recurring tasks guide **design principles** and **verifiable modules**.
- Identified a set of Common **functional modules**

Configuration and Initialisation Module

Field Register and Metadata Module

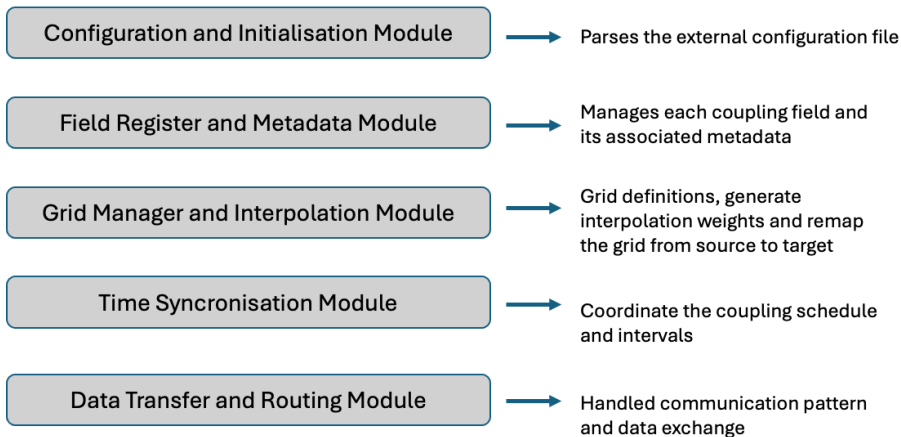
Grid Manager and Interpolation Module

Time Synchronisation Module

Data Transfer and Routing Module

Core Functional Modules

Core Functions of Couplers (Our Analysis)



Core Functional Modules

Module Tasks

Example: Time Synchronisation Module

Module tasks can be extracted as Module contracts

```
/*@  
  requires all_components_reached_time(t);  
  requires clocks_compatible;  
  
  ensures  data_exchanged_at(t);  
  ensures  no_duplicate_or_missed_exchange;  
  
  invariant time_monotonic:  
    time_next >= time_current;  
*/  
void sync_step(double t);
```

Example: Time Synchronisation Module

Module tasks can be extracted as Module contracts

```
/*@  
  requires all_components_reached_time(t);  
  requires clocks_compatible;  
  
  ensures  data_exchanged_at(t);  
  ensures  no_duplicate_or_missed_exchange;  
  
  invariant time_monotonic:  
    time_next >= time_current;  
*/  
void sync_step(double t);
```

- **Pre:** components aligned and clocks consistent.
- **Post:** data exchanged once per step t .
- **Invariant:** simulation time never goes backwards.

Example: Time Synchronisation Module

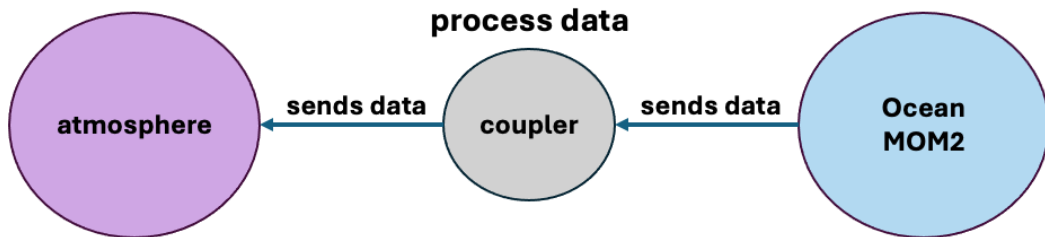
Module tasks can be extracted as Module contracts

```
/*@  
  requires all_components_reached_time(t);  
  requires clocks_compatible;  
  
  ensures data_exchanged_at(t);  
  ensures no_duplicate_or_missed_exchange;  
  
  invariant time_monotonic:  
    time_next >= time_current;  
*/  
void sync_step(double t);
```

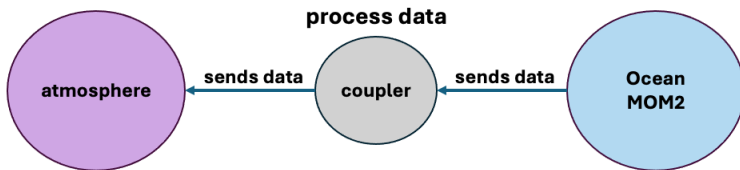
- **Pre:** components aligned and clocks consistent.
- **Post:** data exchanged once per step t .
- **Invariant:** simulation time never goes backwards.
- Can be expressed as contracts using behavioural specification language (e.g., ACSL, JML).
- Enables contract verification of module behaviour.

Simple Coupler Interaction Flow

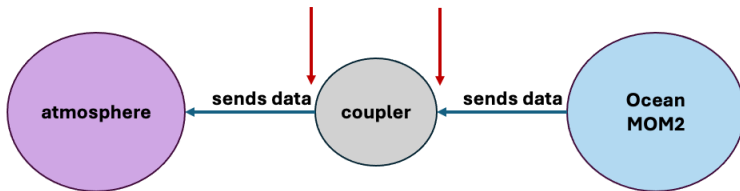
Coupler interaction where the atmosphere and ocean models run independently on their own grids and time steps.



Dynamic Contract Verification

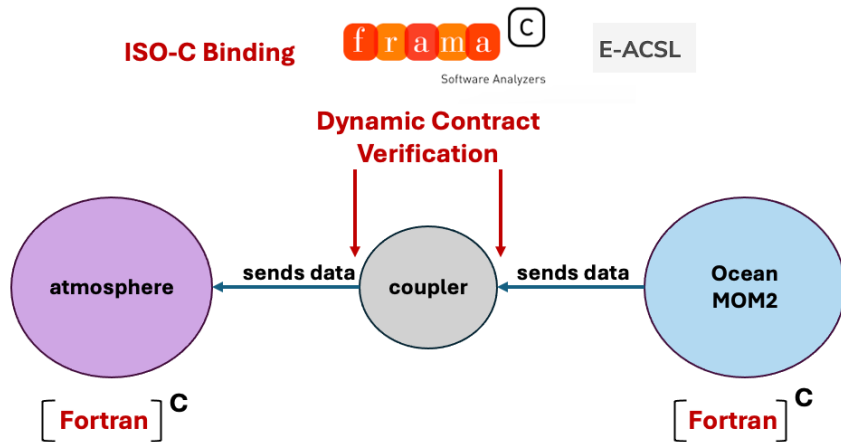


Dynamic Contract Verification



- Runtime assertions to check data flow and consistency at the coupler boundary.
- Poor integration with legacy Fortran models.

Dynamic Contract Verification using Frama-C plugin



Contracts describe and validate expected coupler interactions

Conclusions and Future Work

- Couplers act as the **coordination hub** that keeps atmosphere, ocean, and land models working in sync.
- Couplers share **common behaviors** despite different designs.
- These can be captured as **verifiable contracts** for data exchange, timing, and interpolation.
- Contracts form a basis for clearer **design principles** and modular verification.

Conclusions and Future Work

- Couplers act as the **coordination hub** that keeps atmosphere, ocean, and land models working in sync.
- Couplers share **common behaviors** despite different designs.
- These can be captured as **verifiable contracts** for data exchange, timing, and interpolation.
- Contracts form a basis for clearer **design principles** and modular verification.
- Our approach supports:
 - **Retrofitting existing couplers** with verifiable contracts, and
 - **Designing new couplers from scratch** with formal verification in mind.
- Next steps:
 - Build a minimal contract framework for the existing coupler.
 - Bridge with legacy Fortran systems.
 - Assess reproducibility and performance.

2nd ACM SIGPLAN International Workshop on Programming for the Planet (PROPL 2025)

Thank you!

Towards Modelling and Verification of Coupler Behaviour in Climate Models

Chinmayi Baramashetru
c.baramashetru@kent.ac.uk
University of Kent
UK

Dominic Orchard
dominic.orchard@cl.cam.ac.uk
University of Cambridge
University of Kent
UK

Abstract
Climate models and earth system models often comprise submodels composed via a 'coupler', a software component that enables interaction between submodel components. The continuous exchange of data through couplers creates the risk of subtle errors propagating across components, potentially distorting scientific conclusions. In this paper, we argue

1 Introduction

At the heart of efforts to understand our changing climate are *climate models*—large computational artefacts synthesising centuries of scientific understanding into multi-million line programs from which myriad potential future climatological scenarios are calculated. Like other large-scale software projects built by many people over decades, these models

